

Lecture 8: Lattice Pricing & Dynamic Programming

Leili Rafiee Sevyeri

Based on lecture notes by me and many previous CS335 instructors

Fall 2026

Cheriton School of Computer Science
University of Waterloo

Announcements:

- Assignment one due today at 5pm.
- In-Class Activity #3 grades released.
- Assignment 2 will be released today at 5pm.
- 4th Activity today: first half of lecture v

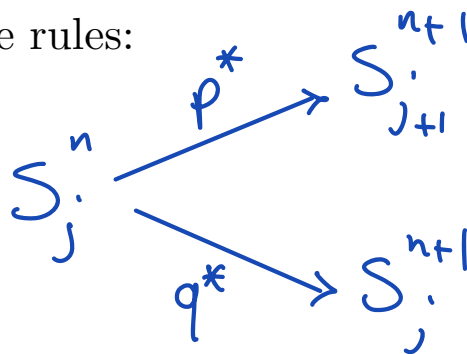
Recap: No Arbitrage Lattice

We saw that based on a risk neutral lattice walk for stock value S , i.e., with risk neutral probabilities defined by

$p^* = \frac{e^{r\Delta t} - e^{-\sigma\sqrt{\Delta t}}}{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}}$, and $q^* = 1 - p^*$ and update rules:

$$S_j^{n+1} = S_j^n e^{-\sigma\sqrt{\Delta t}}$$

$$S_{j+1}^{n+1} = S_j^n e^{+\sigma\sqrt{\Delta t}}$$



The no-arbitrage principle showed us that corresponding option values V on the lattice satisfy:

$$V_j^n = e^{-r\Delta t} (p^* V_{j+1}^{n+1} + q^* V_j^{n+1})$$

i.e., discounted expected value!

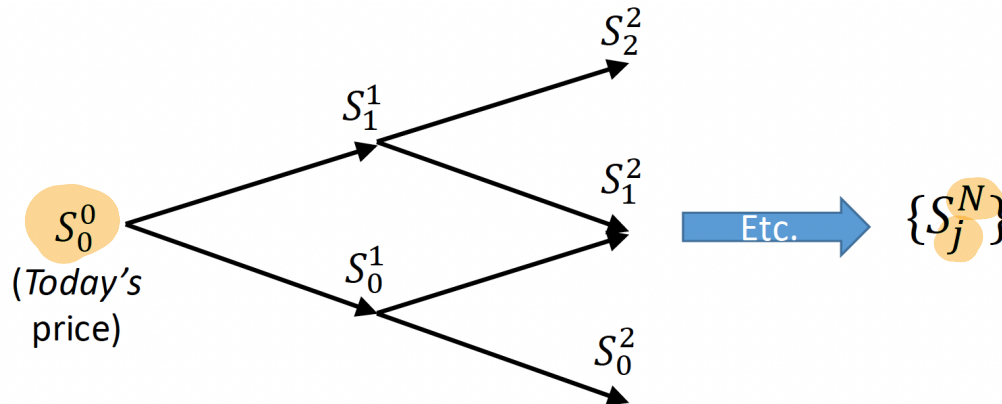
Algorithm: Lattice Pricing for European call option

① Choose number of steps N & step size $\Delta t = T/N$, where $T = \text{expiry time}$.

② Construct N -step tree of stock prices S with:

$$S_j^{n+1} = S_j^n e^{-\sigma\sqrt{\Delta t}}$$

$$S_{j+1}^n = S_j^n e^{\sigma\sqrt{\Delta t}}$$



Each lattice node will also have a corresponding (unknown) option price V_j^n (to be determined...)

Algorithm: Lattice Pricing for European call option

- 3 Assign the step N / time T (i.e., final time) values of option on the tree according to the usual payoff rule (for a call option, in this case) for all j :

$$V_j^N = \max(S_j^N - K, 0)$$

- 4 Work *backwards in time* through the tree layer by layer, using our expected discounted value to find the option value at every node:

Algorithm Working backwards from $N - 1$ to 0

```
1: for  $n = N - 1, \dots, 0$  do
2:   for  $j = 0, \dots, n$  do
3:      $V_j^n = e^{-r\Delta t}(p^*V_{j+1}^{n+1} + (1 - p^*)V_j^{n+1})$ 
4:   end for
5: end for
```

Lattice Expressions

We can express our lattice expressions/updates slightly more concisely:

Stock: Up: $S_{j+1}^{n+1} = u S_j^n$

Down: $S_j^{n+1} = d S_j^n$

Option: $V_j^n = e^{-r\Delta t} (p^* V_{j+1}^{n+1} + (1-p^*) V_j^{n+1})$

where we define: $u = e^{\sigma\sqrt{\Delta t}}$, $d = e^{-\sigma\sqrt{\Delta t}}$, $a = e^{r\Delta t}$

$$p^* = \frac{a-d}{u-d}$$

Notice $S_j^n = S_0 u^j d^{n-j}$ or $S_j^n = S_0 e^{(2j-n)\sigma\sqrt{\Delta t}}$. So one can either determine stock price by direct evaluation or by propagating through the tree.

At each step in the binomial tree, the stock price either:

- goes up by a factor u , or
- goes down by a factor d .

So, after n steps: $S_j^n = S_0 \times u^{\# \text{ up}} \times d^{\# \text{ down}}$

Here,

- n = total number of steps,
- j = number of up moves,
- So the number of down moves is $n-j$.

This gives: $S_j^n = S_0 u^j d^{n-j}$

Also we have: $u = e^{\sigma\sqrt{\Delta t}}$, $d = e^{-\sigma\sqrt{\Delta t}} = \frac{1}{u}$

So $ud=1$. plugging $d=1/u$ into $S_j^n = S_0 u^j d^{n-j}$:

$$S_j^n = S_0 u^j \left(\frac{1}{u}\right)^{n-j} = S_0 u^j u^{-(n-j)}$$

$$= S_0 u^{2j-n}$$

Since $u = e^{\sigma\sqrt{\Delta t}}$: $S_j^n = S_0 e^{(2j-n)\sigma\sqrt{\Delta t}}$

American (call) option lattice pricing

The key difference is that with an American option we can *exercise* the option at any time step, i.e., take the payoff early.

At each lattice step we now have two choices: take the payoff value of the option now OR continue to hold it like we always did in the European case.

We want to act optimally, so we should try to do whatever maximizes the value of the option.

American (call) option lattice pricing

We define at each step the *continuation value* (i.e., value if we do *not* exercise the option, like in European case):

$$(V_j^n)^c = e^{-r\Delta t} (p^* V_{j+1}^{n+1} + (1 - p^*) V_j^{n+1})$$

For an American option, the option payoff at a given lattice step is:

$$\text{payoff} = \max (S_j^n - K, 0)$$

(Notice: this is for any step n , i.e., not just the end step N .)

Thus, the update on the lattice for the actual option value V_j^n under American options is whichever is larger:

$$V_j^n = \max \left((V_j^n)^c, \text{payoff} \right)$$

Implementation comments

This is just a small change to the European lattice pricing algorithm!

Storing the entire tree at once would be $O(N^2)$ storage. Working with data for only one or two columns (i.e., time steps) at a time, you can reduce it to $O(N)$.

Within one time step, your Python code should be *vectorized* for efficiency, such that all entries in a column are computed using operations on vectors. i.e.,

Algorithm Vectorized European Lattice Pricing

```
1: for  $n = N - 1, \dots, 0$  do
2:   [no explicit loops here]
3:   (use vectorized operations on entire column)
4: end for
```

Dynamic Programming

Our American option lattice pricing used the principle of optimality.

For any point on an optimal trajectory, the remaining trajectory is optimal for the corresponding problem initiated at that point.

We'll consider as an example the so-called “marriage problem” (it has various other names)...

The “Marriage Problem”

Imagine you are a terrible person who **only** wants to choose a spouse based on their money.

If you meet and date a sequence of people, how do you know when to marry the current person, vs. break up and date another person in hopes that they are richer?

Problem: Determine a strategy to maximize the *expected wealth* of the person you marry (note: really expected value, not arbitrage).

“Marriage problem” – the rules

- 1 You can meet at most N people, one at a time.
- 2 Each person's wealth is uniformly distributed in $[0, 1]$.
- 3 At step k , you can determine the wealth of the current prospective marriage partner k .
- 4 If you ask k to marry you, they will accept.
- 5 Once married, the game ends. (e.g., divorce not allowed/considered.)

This is an example of optimal stopping

Working Backwards

Start from the final state/person N , and work back to 1.

At step N : you have already not married person $1 \dots N - 1$, so you **must** marry N . (Choosing not to marry at all = 0 wealth, clearly unacceptable!) N 's wealth was drawn uniformly from $[0, 1]$, so their *expected* wealth is 0.5.

At step $N - 1$: Two choices now exist:

- Marry $N - 1$ (whose wealth you know).
- Move on and marry N , whose wealth *you don't yet know!*

What should you do?

Working Backwards

Answer: Use expectation!

Marry $N - 1$ if their (known) wealth $>$ *expected* wealth of N . i.e., if $N - 1$'s wealth > 0.5 .

Then, iterate this idea back to person $k = 1$ to determine what your expected partner's wealth will be, if you always act optimally.

Formal Notation

V_N = expected partner wealth at step N (assuming optimal actions)

W_{N-1} = actual wealth of specific person $N - 1$

$$\bar{W}_{N-1} = E \left[W_{N-1} \mid \underbrace{W_{N-1} > V_N}_{\text{Stopping rule}} \right]$$

| means "conditional on"

= expected wealth of person $N - 1$, assuming you do stop at $N - 1$.

$$p_{N-1} = \text{prob}[W_{N-1} > V_N]$$

Example

Let's work backwards to find the expected wealth of your partner up to person $N - 2$. That is, find V_{N-2} .

Careful: This means the wealth of the person you *ultimately* marry, not necessarily the wealth of person $N - 2$.

Example

Expected partner wealth at step $N-1$, assuming you act optimally, on steps $(N-1, N)$:

$$V_{N-1} = p_{N-1} \overline{W}_{N-1} + (1 - p_{N-1}) V_N$$

Now $V_N = 0.5 = \frac{1+0}{2}$

$$p_{N-1} = \text{prob}(W_{N-1} > 0.5) = 0.5$$

End of Lecture 8

The example is not done!

Example